

Arithmetic Operators

Arithmetic operators are used to perform **mathematical calculations** in a C program.

Operator	Name	Example	Result
+	Addition	10 + 5	15
-	Subtraction	10 - 5	5
*	Multiplication	10 * 5	50
/	Division	10 / 5	2
%	Modulus (Remainder)	10 % 3	1

```
#include <stdio.h>
int main()
{
    int a = 10;
    int b = 3;
    printf("a = %d, b = %d\n", a, b);
    printf("Addition    = %d\n", a + b);
    printf("Subtraction  = %d\n", a - b);
    printf("Multiplication = %d\n", a * b);
    printf("Division     = %d\n", a / b);
    printf("Remainder    = %d\n", a % b);
    return 0;
}
```

a = 10, b = 3
Addition = 13
Subtraction = 7
Multiplication = 30
Division = 3
Remainder = 1

C Operator Precedence Table

Operators	Description	Associativity
() [] -> .	Function call, array, structure/member access	Left → Right
++ --	Post-increment, post-decrement	Left → Right
++ -- + - ! ~ * & sizeof	Pre-increment/decrement, unary operators	Right → Left
* / %	Multiplication, division, modulus	Left → Right
+ -	Addition, subtraction	Left → Right
<< >>	Bitwise shift	Left → Right
< <= > >=	Relational	Left → Right
== !=	Equality	Left → Right
&	Bitwise AND	Left → Right
^	Bitwise XOR	Left → Right
	Bitwise OR	Left → Right
&&	Logical AND	Left → Right
	Logical OR	Left → Right
?:	Conditional / ternary	Right → Left
= += -= *= /= %=	Assignment	Right → Left
,	Comma	Left → Right

Relational Operators

Relational operators are used to **compare two values**. The result of a relational operation is either **1 (true)** or **0 (false)**.

Operator	Meaning	Example	Result
==	Equal to	10 == 5	0
!=	Not equal to	10 != 5	1
>	Greater than	10 > 5	1
<	Less than	10 < 5	0
>=	Greater than or equal to	10 >= 10	1
<=	Less than or equal to	10 <= 5	0

```
#include <stdio.h>
int main()
{
    int a = 10;
    int b = 5;
    printf("a = %d, b = %d\n", a, b);
    printf("a == b : %d\n", a == b);
    printf("a != b : %d\n", a != b);
    printf("a > b : %d\n", a > b);
    printf("a < b : %d\n", a < b);
    printf("a >= b : %d\n", a >= b);
    printf("a <= b : %d\n", a <= b);
    return 0;
}
a = 10, b = 5
a == b : 0
a != b : 1
a > b : 1
a < b : 0
a >= b : 1
a <= b : 0
```

Logical Operators

Logical operators are used to **combine or reverse conditions**. The result is **1 (true)** or **0 (false)**.

Operator	Name	Example	Meaning
&&	Logical AND	a > 5 && b < 10	True if both conditions are true
	Logical OR	a > 5 b < 10	True if any one condition is true
!	Logical NOT	!(a > 5)	Reverses the result

```
#include <stdio.h>
int main()
```

```

{
    int a = 10;
    int b = 5;
    printf("a = %d, b = %d\n", a, b);
    printf("a > 5 && b < 10 : %d\n", a > 5 && b < 10);
    printf("a > 5 || b > 10 : %d\n", a > 5 || b > 10);
    printf("!(a > 5)      : %d\n", !(a > 5));
    return 0;
}
a = 10, b = 5
a > 5 && b < 10 : 1
a > 5 || b > 10 : 1
!(a > 5)      : 0

```

Bitwise Operators

Bitwise operators work **directly on the individual bits** of integer values.

Operator	Name	Example	Meaning
&	Bitwise AND	a & b	1 if both bits are 1
	Bitwise OR	a b	1 if any one of the bits is 1
^	Bitwise XOR	a ^ b	1 if the bits are different
~	Bitwise NOT	~a	Reverses every bit
<<	Left Shift	a << 1	Shifts bits to the left
>>	Right Shift	a >> 1	Shifts bits to the right

```

#include <stdio.h>
int main()
{
    int a = 5;
    int b = 3;
    printf("a = %d, b = %d\n", a, b);
    printf("a & b = %d\n", a & b);
    printf("a | b = %d\n", a | b);
    printf("a ^ b = %d\n", a ^ b);
    printf("~a    = %d\n", ~a);
    printf("a << 1 = %d\n", a << 1);
    printf("a >> 1 = %d\n", a >> 1);
    return 0;
}
a = 5, b = 3
a & b = 1
a | b = 7

```

```
a ^ b = 6
~a    = -6
a << 1 = 10
a >> 1 = 2
```

Assignment Operators

Assignment operators are used to **assign a value to a variable**.

Operator	Example	Equivalent Expression	Meaning
=	a = 10	—	Assigns 10 to a
+=	a += 5	a = a + 5	Adds 5 to a
-=	a -= 5	a = a - 5	Subtracts 5 from a
*=	a *= 5	a = a * 5	Multiplies a by 5
/=	a /= 5	a = a / 5	Divides a by 5
%=	a %= 5	a = a % 5	Assigns remainder to a
&=	a &= 5	a = a & 5	Bitwise AND assignment
^=	a ^= 5	a = a ^ 5	Bitwise XOR assignment
<<=	a <<= 2	a = a << 2	Left-shift and assign
>>=	a >>= 2	a = a >> 2	Right-shift and assign

```
#include <stdio.h>
int main()
{
    int a = 10;
    printf("Initial value of a = %d\n", a);
    a += 5;
    printf("After a += 5 : %d\n", a);
    a -= 3;
    printf("After a -= 3 : %d\n", a);
    a *= 2;
    printf("After a *= 2 : %d\n", a);
    a /= 4;
    printf("After a /= 4 : %d\n", a);
    a %= 3;
    printf("After a %%= 3 : %d\n", a);
    return 0;
}
```

Initial value of a = 10

After a += 5 : 15

After a -= 3 : 12

After a *= 2 : 24

After a /= 4 : 6

After a %= 3 : 0

Conditional Operator

The **conditional operator** is also called the **ternary operator** because it works with **three operands**.

It is used to make a **simple decision** and choose one of two values.

Operator	Example	Meaning
?:	Result = (a > b)? a : b;	If condition is true, choose a; otherwise choose b

```
#include <stdio.h>
int main()
{
    int a = 10;
    int b = 20;
    int max;
    max = (a > b) ? a : b;
    printf("a = %d\n", a);
    printf("b = %d\n", b);
    printf("Largest value = %d\n", max);
    return 0;
}
a = 10
b = 20
Largest value = 20
```

Input / Output Functions

Input/Output (I/O) functions are used to **take input from the user** and **display output on the screen**.

Function	Type	Example	Purpose
printf()	Output	printf("%d", a);	Displays output on the screen
scanf()	Input	scanf("%d", &a);	Reads input from the user
getchar()	Input	ch = getchar();	Reads a single character
putchar()	Output	putchar(ch);	Displays a single character
puts()	Output	puts("Hello");	Displays a string
fgets()	Input	fgets(name, 30, stdin);	Reads a string including spaces

Format Specifiers Table

Data Type	scanf()	printf()	Example
int	%d	%d	int age = 20;
float	%f	%f	float mark = 85.5;
double	%lf	%f	double salary = 50000.75;
char	%c	%c	char grade = 'A';
long	%ld	%ld	long population = 1000000;

```

#include <stdio.h>
int main()
{
    int age;
    float mark;
    double salary;
    char grade;
    long population;
    printf("Enter age: ");
    scanf("%d", &age);
    printf("Enter mark: ");
    scanf("%f", &mark);
    printf("Enter salary: ");
    scanf("%lf", &salary);
    printf("Enter grade: ");
    scanf(" %c", &grade);
    printf("Enter population: ");
    scanf("%ld", &population);
    printf("\n--- Output ---\n");
    printf("Age      = %d\n", age);
    printf("Mark      = %.2f\n", mark);
    printf("Salary     = %.2f\n", salary);
    printf("Grade      = %c\n", grade);
    printf("Population = %ld\n", population);
    return 0;
}

```

Enter age: 20
 Enter mark: 85.5
 Enter salary: 50000.75
 Enter grade: A
 Enter population: 1000000

--- Output ---
 Age = 20
 Mark = 85.50
 Salary = 50000.75
 Grade = A
 Population = 1000000

```

#include <stdio.h>
int main()
{
    char ch;
    char name[50];
    printf("Enter a character: ");
    ch = getchar();
    printf("You entered: ");
    putchar(ch);
    getchar(); // consumes the Enter key
}

```

```

printf("\nEnter your name: ");
fgets(name, sizeof(name), stdin);
printf("\nName: ");
puts(name);
return 0;
}

```

Enter a character: A

You entered: A

Enter your name: Ashok Rajan

Name: Ashok Rajan

Built-in Functions

In C, **built-in functions** generally refer to **predefined/library functions** provided by the C standard libraries. We can directly use these functions instead of writing the same functionality ourselves.

To use a library function, we usually include the required **header file** using `#include`

Function	Header File	Purpose	Example
<code>sqrt()</code>	<code><math.h></code>	Finds square root	<code>sqrt(25) → 5</code>
<code>pow()</code>	<code><math.h></code>	Finds power	<code>pow(2, 3) → 8</code>
<code>abs()</code>	<code><stdlib.h></code>	Finds absolute value	<code>abs(-10) → 10</code>
<code>strlen()</code>	<code><string.h></code>	Finds string length	<code>strlen("Hello") → 5</code>
<code>toupper()</code>	<code><ctype.h></code>	Converts character to uppercase	<code>toupper('a') → A</code>
<code>tolower()</code>	<code><ctype.h></code>	Converts character to lowercase	<code>tolower('A') → a</code>
<code>isalpha()</code>	<code><ctype.h></code>	Checks whether character is alphabet	<code>isalpha('A') → non-zero</code>
<code>isdigit()</code>	<code><ctype.h></code>	Checks whether character is digit	<code>isdigit('5') → non-zero</code>

```

#include <stdio.h>
#include <math.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
int main()
{
    char ch = 'a';
    char str[] = "Hello";
    printf("Square root of 25 = %.2f\n", sqrt(25));
    printf("2 raised to power 3 = %.2f\n", pow(2, 3));
    printf("Absolute value of -10 = %d\n", abs(-10));
    printf("Length of Hello = %zu\n", strlen(str));
    printf("Uppercase of a = %c\n", toupper(ch));
    printf("Lowercase of A = %c\n", tolower('A'));
    printf("Is 'A' an alphabet? %d\n", isalpha('A'));
    printf("Is '5' a digit? %d\n", isdigit('5'));
    return 0;
}

```

Square root of 25 = 5.00
2 raised to power 3 = 8.00
Absolute value of -10 = 10
Length of Hello = 5
Uppercase of a = A
Lowercase of A = a
Is 'A' an alphabet? 1024
Is '5' a digit? 2048